documents

March 29, 2026

1 Documents Notebook

For this chatbot I will be using bash man pages, tealdeer output and the output from the bash help system for builtin commands.

There will need to be 2 pipelines. - The first will be the data ingestion pipeline, which is in this notebook. - Then we will need a query pipeline that will use our domain specific embeddings as the context for an LLM prompt. This is the RAG implementation that will be built. It is located in bashbot.ipynb and the associated script.

The dependencies are in requirements.txt.

For the default Ollama implementation, you will need to install Ollama and pull the following models:

- *nomic-embed-text*
- *llama3*

1.0.1 Data Generation

- Generate man pages for all known commands (from bash)

  ```
  compgen -c | sort | uniq -d | xargs -I {} sh -c "man '{}' > '{}.man'"
  2>>noman.txt
  ```

- Delete empty files (no man page)

  ```
  find . -type f -empty -delete
  ```

- Use tealdeer for additional data

  ```
  tldr --update
  ```

- Generate tldr files

  ```
  compgen -c | sort | uniq -d | xargs -I {} sh -c "tldr '{}' > '{}.tldr'"
  2>>notldr.txt
  ```

- Delete empty files (no tldr entry)

  ```
  find . -type f -empty -delete
  ```

- Replace {COMMAND} with to avoid LLM errors with reserved word structure.

  ```
  find ./ -type f -name "*.txt" -exec sed -i -e 's/{COMMAND}/<CMD>/g' {} \;
  ```

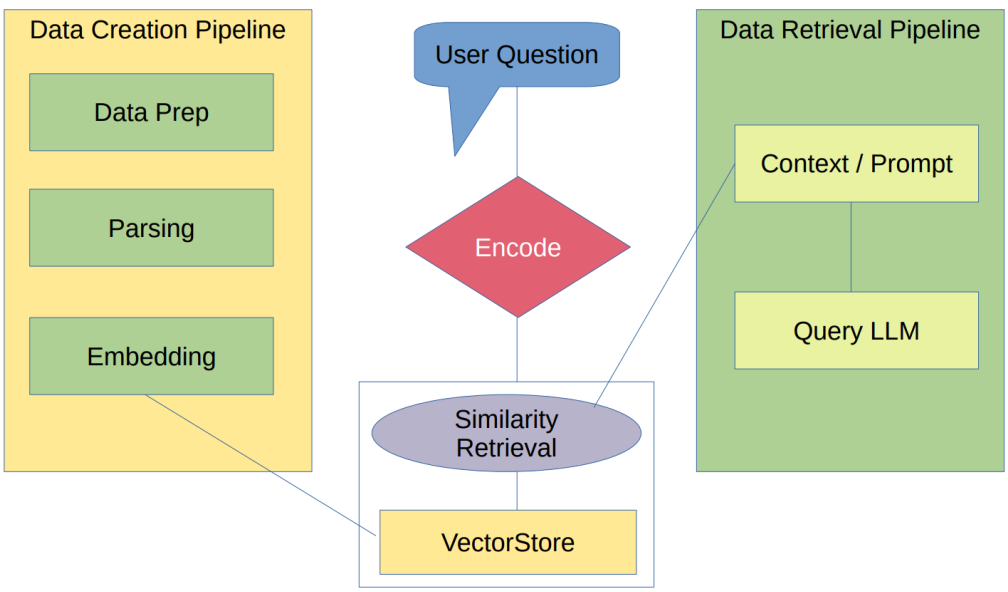
NOTE I switched to shellscript man2txt.sh to get the man pages into utf-8 and remove the control characters with mandoc. And I did not capture the command pipeline I used to capture the shell builtins, but it was a similar construct to the compgen pipelines.

1.0.2 man2txt.sh

```bash
#!/bin/bash

while IFS= read -r cmd; do
   mandoc -mdoc -c -T utf8 "$(man -w ${cmd})" | col -b > "$cmd".txt 2>error.err
done < <(compgen -c | sort | uniq)
find . -type f -empty -delete
```

1.0.3 Architecture



1.1 Data Ingestion Pipeline for Bashbot

1.1.1 Imports

[1]:
```
import os
import sys
import shutil
from pathlib import Path
from langchain_core.documents import Document
from langchain_community.document_loaders import DirectoryLoader, TextLoader␣
↪#,PyPDFLoader, PyMuPDFLoader for PDFs in the future
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

1.1.2 NOTE

If you want to use the existing vector db and skip parsing the docs, set the **USE_EXISTING=True** below

[2]:
```
USE_EXISTING=False  #  If True, new files will not be added to the vector store.
 ↪
INIT=True           # If True, te vector store will be deleted and data files␣
 ↪will be proessed.
VERBOSE = True      # If True, additional output is generated.
```

1.1.3 To remove the exiting data store set INIT=True above.

Chromadb is thread-safe but not process safe. The method call in the VectorStore class will generate an error if used within the same process. To avoid this, you can clean the existing vectorstore by manually removing the ../data/vector_store directory or by setting INIT=True above.

1.2 Class Definitions

I want the flexibility to have pluggable implementations. I am going to wrap this part in classes. Inspiration for this came from Krish Naik's YouTube video https://youtu.be/o126p1QN_RI?si=7xC5H47A3iuu52RK and pixegami https://youtu.be/2TJxpyO3ei4?si=zPNEsAFWWy5Cenzq

1.2.1 Loader

Processes data files and handles the chunking. This notebook is basically a test harness. This code is the canonical representation of loader.py used by the CLI.

[3]:
```
class Loader:
    """ This class handles loading of the documents """

    def __init__(self, path_name: str = "../data", glob: str = "**/*.txt",␣
 ↪chunk_size=500, chunk_overlap=100):
        """ """
        self.documents = []
        self.path = Path(path_name)
        self.glob = glob
        self.chunks = None
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap

    def _split_documents(self):
        """Chunks the text into chunk_size with an overlap specified"""

        text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=self.chunk_size,
            chunk_overlap=self.chunk_overlap,
            length_function=len,
```

```
            separators=["\n\n", "\n", " ", ""]
        )
        self.chunks = text_splitter.split_documents(self.documents)
        print(f"Split {len(self.documents)} documents into {len(self.chunks)}␣
↪chunks.")

    def process_files(self):
        """Processes all files in the path"""

        files = list(self.path.glob(self.glob))
        print(f"Preparing to process {len(files)} text files...")

        for f in files:
            try:
                loader = TextLoader(str(f))
                docs = loader.load()

                for d in docs:
                    d.metadata['manpage'] = Path(f).stem
                    d.metadata['source_file'] = f.name
                    d.metadata['file_type'] = 'txt'
                    d.metadata['platform'] = 'linux'
                    d.metadata['source'] = 'manpage'

                    self.documents.extend(docs)

            except Exception as e:
                print(f"    ERROR {e}")

        print(f"Total documents loaded: {len(self.documents)}.")
        self._split_documents()
```

1.2.2 EmbeddingMgr

This code is the canonical representation of embeddings.py used by the CLI.

[4]:
```
import numpy as np
import chromadb
import ollama
from typing import List, Dict, Any, Tuple
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_ollama import OllamaEmbeddings
from langchain_classic.chains import RetrievalQA
from langchain_community.llms import Ollama
```

```python
class EmbeddingMgr:
    """This class manages the embedding implementation.
    """

    def __init__(self, embedding_model_name: str = "nomic-embed-text") ->␣
↪List[Any]:
        """
        ctor

        Args:
            model_name: embedding model name for embeddings.

        Returns:
            List of embeddings
        """

        self.embedding_model_name = embedding_model_name
        self.model = None
        self._load_components()

    def _load_components(self):
        """This method loads the embedding model defined in the ctor."""
        try:
            print(f"Loading Embedding model {self.embedding_model_name}.")
            self.model = OllamaEmbeddings(model=self.embedding_model_name)
            print(f"Embedding Model {self.model} loaded successfully.")
        except Exception as e:
            print(f"Exception while loading {self.embedding_model_name}: {e}")
            raise

    def add_embeddings(self, texts):
        """
        Generate embeddings for a list of text strings

        Args:
            texts: List of strings
        """
        if not self.model:
            raise ValueError("Model has not been loaded.")

        print(f"Generating embeddings for {len(texts)} strings.")

        embeddings = []
        for chunk in texts:
        # generate embeddings
            embedding = ollama.embeddings(
                model= self.embedding_model_name,
```

```
                prompt=chunk.page_content
            )["embedding"]

            embeddings.append(embedding)

        return embeddings
```

1.2.3 VectorStore

This code is the canonical representation of vectorstore.py used by the CLI.

[5]:
```
import chromadb
import uuid
from chromadb.config import Settings
from chromadb.utils.batch_utils import create_batches

class VectorStore:
    """Manages embeddings in a vector database"""

    def __init__(self, collection_name: str = "bashdocs", persist_directory:␣
↪str = "../data/vector_store"):
        """
        ctor

        Args:
            collection_name: Name of the collection
            persist_directory: Path to directory for persistant vectors.
        """
        self.collection_name = collection_name
        self.persist_directory = persist_directory
        self.client = None
        self.collection = None
        self._initialize_store()

    def _initialize_store(self):
        """Initialize the ChromaDB client and collection"""
        try:
            # Create the persistent ChromaDB client
            os.makedirs(self.persist_directory, exist_ok=True)
            self.client = chromadb.PersistentClient(path=self.persist_directory)

            # Get or create the collection
            self.collection = self.client.get_or_create_collection(
                name=self.collection_name,
                metadata={"description": "Bash commands document embeddings for␣
↪RAG"}
            )
```

```
            print(f"Vector store initialized for collection {self.
↪collection_name}")
            print(f"Exisiting documents in collection: {self.collection.
↪count()}")

        except Exception as e:
            print(f"Error initializing vector store: {e}")
            raise

    def add_documents(self, documents: List[Any], embeddings: np.ndarray):
        """
        Add documents to the vector store

        Args:
            documents: List of langchain Documents
            embeddings: The embeddings for the documents
        """
        if len(documents) != len(embeddings):
            raise ValueError("Number of documents to embeddings mismatch.")

        print(f"Adding {len(documents)} documents to the vector store...")

        # Prepare data for ChromaDB
        ids = []
        metadatas = []
        documents_text = []
        embeddings_list = []

        for idx, (doc, embedding) in enumerate(zip(documents, embeddings)):
            # Generate the uuid
            doc_id = f"doc_{uuid.uuid4().hex[:8]}_{idx}"
            ids.append(doc_id)

            # Metadata
            metadata = dict(doc.metadata)
            metadata['doc_index'] = idx
            metadata['content_length'] = len(doc.page_content)
            metadatas.append(metadata)

            # Document Page Content
            documents_text.append(doc.page_content)

            # Embedding vetor
            embeddings_list.append(embedding)

        # Add it to the collection
        try:
```

```
            max_batch = 5000 # batch max is 5461
            for i in range(0, len(embeddings), max_batch):
                self.collection.add(
                    embeddings=embeddings_list[i:i + max_batch],
                    documents=documents_text[i:i + max_batch],
                    ids=ids[i:i + max_batch],
                    metadatas=metadatas[i:i + max_batch]
                )
            print(f"Successfully added {len(documents)} documents to the store.
↪")
            print(f"Total documents in collection: {self.collection.count()}")

        except Exception as e:
            print(f"Error adding documents to vector store: {e})")
            raise
```

1.3 Notebook Exploration

Determine operting mode for the vector store and remove the store if it is being initalized. This is similar to running the CLI with the -d switch.

[6]:
```
if INIT and not USE_EXISTING:
    store_path = Path("../data/vector_store/")
    print(f"Removing vector store at {store_path}.")
    shutil.rmtree(store_path, ignore_errors=True)
```

```
Removing vector store at ../data/vector_store.
```

1.3.1 Ollama Version

This project was prototyped with Grok using a trial API key. The original vision was a local RAG implementation and it has been converted to use Ollama models locally.

Initialize the Embedding Manager

Controls the generation of embedding vectors using the model passed in the ctor.

[7]:
```
emb_mgr = EmbeddingMgr("nomic-embed-text")
emb_mgr
```

```
Loading Embedding model nomic-embed-text.
Embedding Model model='nomic-embed-text' validate_model_on_init=False
base_url=None client_kwargs={} async_client_kwargs={} sync_client_kwargs={}
mirostat=None mirostat_eta=None mirostat_tau=None num_ctx=None num_gpu=None
keep_alive=None num_thread=None repeat_last_n=None repeat_penalty=None
temperature=None stop=None tfs_z=None top_k=None top_p=None loaded successfully.
```

[7]:
```
<__main__.EmbeddingMgr at 0x7fcca014a4e0>
```

Initialize the Vector Store

Currently only supports ChromaDB.

[8]:
```
vector_store = VectorStore()
vector_store
```

```
Vector store initialized for collection bashdocs
Exisiting documents in collection: 0
```

[8]:
```
<__main__.VectorStore at 0x7fcc3d07dfa0>
```

[9]:
```
#import ollama
#import chromadb
#from langchain_text_splitters import RecursiveCharacterTextSplitter
```

1.3.2 Create and Add embeddings

if USE_EXISTING = False then we read in the files from the data directory and add them to the vector store. This will cause duplicates if run multiple times without setting INIT=True to clean the db store.

[10]:
```
# get the text from the documents and create embeddings to store
if not USE_EXISTING:
    loader = Loader()
    loader.process_files()
    if VERBOSE:
        print(loader.chunks[3])

    texts = [doc for doc in loader.chunks]
    print(type(texts[0]))
    # Use the EmbeddingMgr class to generate them
    embeddings = emb_mgr.add_embeddings(texts)

    # Add the documents to the vector store
    vector_store.add_documents(loader.chunks, embeddings)
```

```
Preparing to process 2077 text files…
Total documents loaded: 2077.
Split 2077 documents into 34847 chunks.
page_content='conventional yacc yy-prefixed names to whatever the -p option may
specify.
The code file, e.g., y.tab.c is modified to #include this file as well as the
y.tab.h file, enforcing consistent usage of the symbols defined in those
files.  The supplementary header file makes it simpler to separate compilation
of lex- and yacc-files.  If the option is not specified, will insert #line
directives in the generated code.  The #line directives let the C compiler'
metadata={'source': 'manpage', 'manpage': 'yacc', 'source_file': 'yacc.txt',
'file_type': 'txt', 'platform': 'linux'}
<class 'langchain_core.documents.base.Document'>
Generating embeddings for 34847 strings.
```

```
Adding 34847 documents to the vector store…
Successfully added 34847 documents to the store.
Total documents in collection: 34847
```

1.4 Prompt and RAG Retrieval Pipeline

Support functions to produce queries for testing and evaluation.

[11]:
```
def get_relavent_docs(query, embedding_manager, vector_store, top_k=3):
    # Get embedding for question
    question_embedding = ollama.embeddings(
        model=embedding_manager.embedding_model_name,
        prompt=query
    )["embedding"]

    # Find relevant chunks
    results = vector_store.collection.query(
        query_embeddings=[question_embedding],
        n_results=top_k
    )
#    print(results)
    return results
```

[12]:
```
def send_question(query, llm, embedding_manager, vector_store, top_k=3, pure:␣
↪bool = False) -> Tuple:
    contex = ""
    found = 0

    if not pure:
        results = get_relavent_docs(query, embedding_manager, vector_store,␣
↪top_k)
        found = results["ids"]
        if len(found[0]) > 0:
            # Build context from retrieved chunks
            context = "\n\n".join(results["documents"][0])
        else:
            return ("No relavent documents found.", 0)

    prompt = f"""You are a professional AI assistant that specializes in␣
↪answering
        questions about Linux commands.
        Use ONLY the following context to answer the question accurately and␣
↪concisely.

        Context:
        {context}

        Question: {query}
```

```
        Answer:"""

    # generate answer
    response = ollama.chat(
        model=llm,
        messages=[{
            "role": "user",
            "content": prompt
        }]
    )

    return (found, response["message"]["content"])
```

[13]:
```
found, result = send_question("how do you delete files?", "llama3", emb_mgr,␣
↪vector_store)
print(f"Found {len(found[0])} documents:\n{found}\n{result}")
```

```
Found 3 documents:
[['doc_702430b5_20016', 'doc_adccd534_11664', 'doc_1783069a_1960']]
To delete files, you would use the `find` command followed by the `-delete`
action.
```

1.5 Testing

Read the JSON questions and answers from the test_questions.json (converted from the UTVerse generated data).

Test each question from the JSON file of the top Linux questions generated by UTVerse and capture the LLM answer. Then compare the LLM answer to the answer provided by the chatbot. Looking at the cosine similarity may provide a general evaluation, but a manual inspection will probably be needed for accurate assessment.

1.5.1 submit a single question and return the answer

[14]:
```
import json
# from langchain_groq import ChatGroq
from langchain_ollama import OllamaLLM
from langchain_community.chat_models import ChatOllama
from langchain_core.messages import HumanMessage, SystemMessage
```

1.5.2 Load the JSON test data

[15]:
```
with open("../test/test_questions.json", "r") as jsf:
    test_data = json.load(jsf)
```

[16]:
```
type(test_data)
```

[16]: dict

[17]:
```
import pandas as pd

df_test = pd.DataFrame(test_data)
```

[18]:
```
df_test.head()
```

[18]:
```
                                      linux_questions
0  {'id': 1, 'question': 'What is Linux?', 'answe…
1  {'id': 2, 'question': 'What is a Linux distrib…
2  {'id': 3, 'question': 'What is the difference …
3  {'id': 4, 'question': 'What is the Linux shell…
4  {'id': 5, 'question': 'How do you check the cu…
```

[19]:
```
results = []
for item in test_data['linux_questions'][:20]:
#    result = rag_call(item['question'], fido, llm)
# send_question(query, llm, embedding_manager, vector_store, top_k=3, pure:␣
↪bool = False)
    found, result = send_question(item['question'], "llama3", emb_mgr,␣
↪vector_store)
    item['result'] = result
    results.append(item)
```

[20]:
```
len(results)
```

[20]: 20

[21]:
```
df_results = pd.DataFrame(results)
```

[22]:
```
df_results.head()
```

[22]:
```
   id                                          question  \
0   1                                     What is Linux?
1   2                       What is a Linux distribution?
2   3  What is the difference between the kernel and …
3   4                            What is the Linux shell?
4   5    How do you check the current working directory?

                                                 answer  \
0  Linux is an open-source, Unix-like operating s…
1  A Linux distribution (distro) bundles the Linu…
2  The kernel manages hardware/resources (CPU, me…
3  The shell is a command interpreter (e.g., bash…
4                                        Use 'pwd'.

                                                result
```

```
0  Based on the provided context, I'll do my best…
1  Based on the provided context, I'll provide an…
2  Based on the provided context, there is no exp…
3  According to the provided context, a Linux she…
4  A question about Linux commands!\n\nAccording …
```

[23]:
```
df_results['result'][0]
```

[23]:
```
'Based on the provided context, I\'ll do my best to answer your question.
However, I must point out that the given context doesn\'t seem to be directly
related to the topic of "What is Linux?" Nevertheless, I can try to provide an
accurate and concise answer within the limitations of the provided
context.\n\nFrom what I see, the context appears to be related to a menu or
options for a command. The presence of DVD and SVCD suggests that the context
might be related to multimedia or video formats. Given this information, my
attempt at answering your question would be:\n\n"Linux is not explicitly
mentioned in this context, but it seems to relate to commands or options for
processing multimedia files like DVDs and SVCDs."\n\nPlease note that this
answer is an educated guess based on the provided context, as the actual context
does not provide direct information about what Linux is.'
```

1.6 Function Definitions

Helper to calculate cosine similarity between the provided JSON answer and the LLM answer

[24]:
```
def gen_embeddings(text, embedding_manager) -> str:
    # Generate embeddings for a given str
    return ollama.embeddings(
        model=embedding_manager.embedding_model_name,
        prompt=text
    )["embedding"]
```

[25]:
```
import numpy as np
from numpy.linalg import norm

def calculate_similarity(x, y):
    """Calculates the cosine similarity between 2 embeddings"""
    cosine = np.dot(x, y) / (norm(x) * norm(y))
    return cosine
```

Test Loop

[26]:
```
def test_similarity(data: Dict = None, iterations: int = 20, pure: bool =␣
 ↪False, top_k: int = 3) -> List[Dict]:
    results = []

    for item in data['linux_questions'][:iterations]:
```

```
        found, result = send_question(item['question'], "llama3", emb_mgr,␣
↪vector_store, pure=pure)
        X = gen_embeddings(item['answer'], emb_mgr)
        Y = gen_embeddings(result, emb_mgr)
        similarity = calculate_similarity(X, Y)
        item['result'] = result
        item['similarity'] = similarity
#        item['context'] = result[2]
        results.append(item)


    return results
```

[27]:
```
results = test_similarity(test_data, pure=False)
```

Average Similarity (with RAG)

[28]:
```
df = pd.DataFrame(results)
df.head()
print(df['similarity'].mean())
```

```
0.7278128461888962
```

[29]:
```
df['similarity'].describe()
```

[29]:
```
count    20.000000
mean      0.727813
std       0.110033
min       0.491928
25%       0.651041
50%       0.742844
75%       0.814304
max       0.901636
Name: similarity, dtype: float64
```

[30]:
```
df['similarity'].value_counts()
```

[30]:
```
similarity
0.689536    1
0.795996    1
0.781816    1
0.901636    1
0.586233    1
0.491928    1
0.657903    1
0.828857    1
0.816009    1
0.791524    1
```

```
0.676350    1
0.613295    1
0.649415    1
0.605803    1
0.730991    1
0.822184    1
0.896764    1
0.813735    1
0.754698    1
0.651583    1
Name: count, dtype: int64
```

1.6.1 Test without RAG

[31]:
```
results = test_similarity(test_data, pure=False)
```

Average Similarity (without RAG)

[32]:
```
df = pd.DataFrame(results)
df.head()
print(df['similarity'].mean())
```

```
0.7348923658483
```

[33]:
```
df['similarity'].describe()
```

[33]:
```
count    20.000000
mean      0.734892
std       0.111022
min       0.480363
25%       0.679134
50%       0.746652
75%       0.826387
max       0.893483
Name: similarity, dtype: float64
```

[34]:
```
df['similarity'].value_counts()
```

[34]:
```
similarity
0.717942    1
0.842055    1
0.814583    1
0.851137    1
0.596113    1
0.480363    1
0.691449    1
0.844255    1
0.834207    1
0.795741    1
```

```
0.683082    1
0.529046    1
0.735571    1
0.651726    1
0.764287    1
0.724003    1
0.893483    1
0.823780    1
0.757733    1
0.667291    1
Name: count, dtype: int64
```

1.6.2 Test with RAG and *top_k=2*

[35]:
```
results = test_similarity(test_data, top_k=2, iterations=20, pure=False)
```

[36]:
```
df = pd.DataFrame(results)
df.head()
print(df['similarity'].mean())
```

```
0.7510102123275955
```

[37]:
```
df['similarity'].describe()
```

[37]:
```
count    20.000000
mean      0.751010
std       0.117239
min       0.489568
25%       0.664631
50%       0.770166
75%       0.842650
max       0.905250
Name: similarity, dtype: float64
```

[38]:
```
df['similarity'].value_counts()
```

[38]:
```
similarity
0.789976    1
0.807991    1
0.831876    1
0.905250    1
0.614543    1
0.489568    1
0.664694    1
0.842030    1
0.868325    1
0.813180    1
0.672107    1
```

```
0.605694    1
0.692332    1
0.625679    1
0.742680    1
0.895131    1
0.899839    1
0.844512    1
0.750357    1
0.664441    1
Name: count, dtype: int64
```

[39]:
```
results = test_similarity(test_data,␣
↪iterations=len(test_data['linux_questions']), pure=False)
```

[40]:
```
len(results)
```

[40]:
```
50
```

[41]:
```
df = pd.DataFrame(results)
df.head()
print(df['similarity'].mean())
```

```
0.737019449847692
```

[42]:
```
df['similarity'].describe()
```

[42]:
```
count    50.000000
mean      0.737019
std       0.090254
min       0.529801
25%       0.695734
50%       0.748721
75%       0.803341
max       0.898194
Name: similarity, dtype: float64
```

[43]:
```
df['similarity'].value_counts()
```

[43]:
```
similarity
0.742581    1
0.738101    1
0.798010    1
0.862541    1
0.591144    1
0.529801    1
0.716255    1
0.844059    1
0.816009    1
```

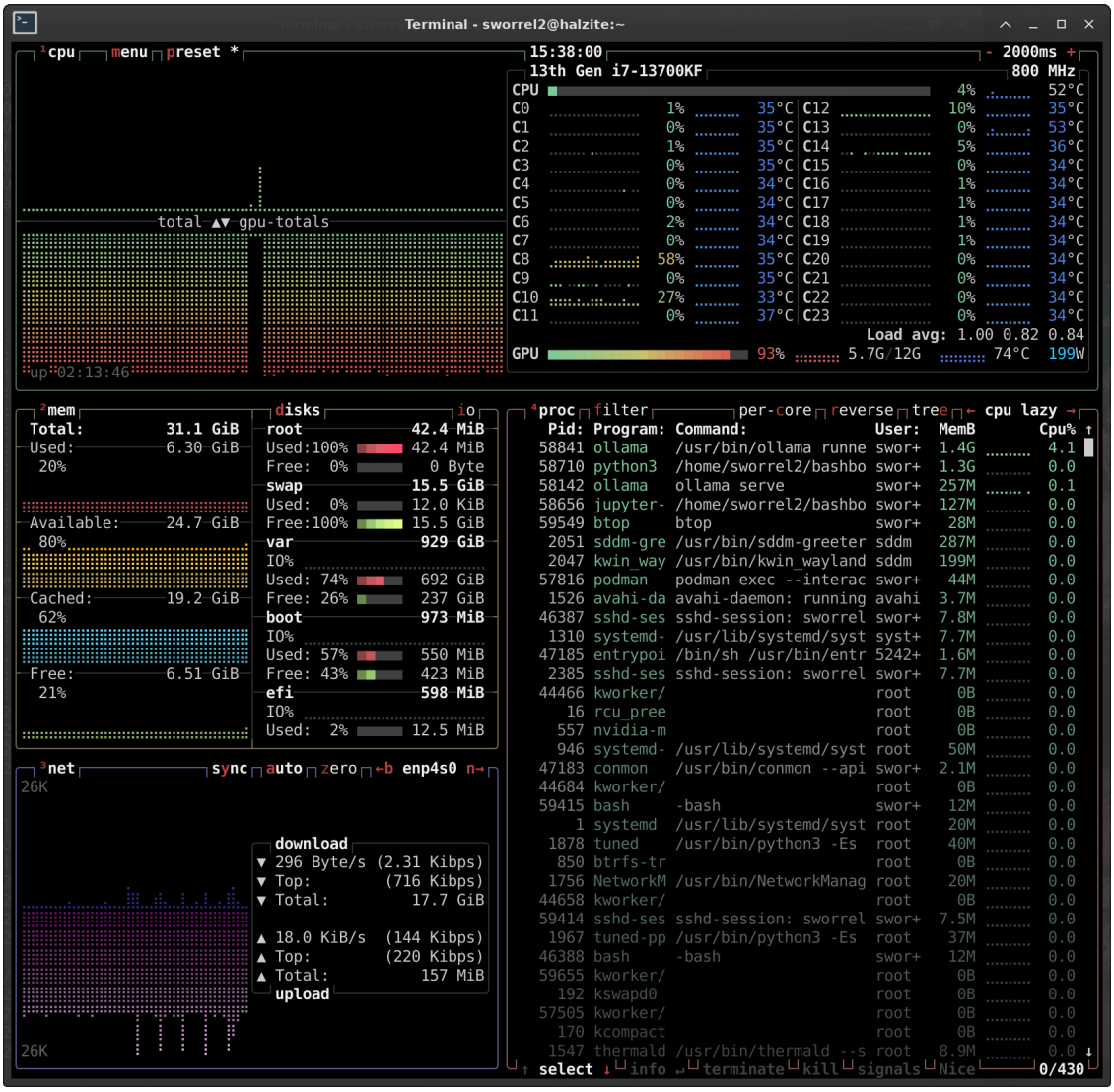
```
0.803527    1
0.664664    1
0.611892    1
0.744439    1
0.590784    1
0.778609    1
0.878514    1
0.898194    1
0.840917    1
0.668133    1
0.699404    1
0.830964    1
0.696940    1
0.803869    1
0.815909    1
0.748027    1
0.561003    1
0.765857    1
0.625778    1
0.609731    1
0.767060    1
0.732703    1
0.641628    1
0.695332    1
0.753747    1
0.613969    1
0.743265    1
0.778375    1
0.762072    1
0.722898    1
0.749415    1
0.546283    1
0.780069    1
0.764941    1
0.836948    1
0.746551    1
0.770181    1
0.848052    1
0.802781    1
0.816317    1
0.702730    1
Name: count, dtype: int64
```

1.7 Conclusions

Choosing models is important for RAG performance. I was able to get comparable results between Grok and the llama3 local LLM. The processing of the input documents was performed locally , however chunking allowed the GPU utilization to be relatively steady at 12%-30% on my Nvidia

4070.

The real GPU utilization comes from the LLM model processing a query.



Note the GPU spike as well as the jump in wattage required from 42.7 for chunk processing to 199 for LLM processing.

1.8 Future Direction

I began with Grok since it is easy to generate an API key, however my main goal is to use a local Ollama model and only return results from the vector_store. I have installed Ollama and am running it as a systemd servie on both Debian 13 (Trixie). My end goal is to create a docker container that can run efficiently on CPU in an OCI container. This would allow me to deploy a container to a server and allow users to import additional documents into the existing store to share with the development team.

To support multiple document formats, I intened to integrate Docling into the input file processing pipeline. This will enable robust support for supported formats without having to juggle a bunch of implementations to import word docs, pdfs, etc…

Once that is completed, I will create a web UI using OpenWebUI to create the UX and allow the

specification of personality, as well as the system prompts to enable finer control over output from various users. Eventually users will be able to upload documents to add to the store on the host machine to provide a secure central LLM focused on the source code, documents and database artifacts for a single user or specific team. The data that matetrs in my daily work involves data involving these artifacts and I hope to have a small focused LLM that is more accurate at generating meaningful answers to my questions during the design of new project architecture.

I rebuilt my dev setup to be able to work on my laptop, but ssh into my desktop system to run the models and utilize my GPU. For future versions I will be converting the projct to a literate version using Emacs orgmode and TRAMP to have the documentation and the source code in a single text file with control to spin up containers to run Ollama as well as Jupyter kernels on the GPU host to allow remote exploratory work to be continues on a "live" system. To complete the full vision will require a new Lisp package I am working on to supplement ein and tramp to allow control over the entire dev pipeline from my Emacs environment.

1.8.1 References

IBM Technology LangChain vs LangGraph: A Tale of Two Frameworks https://youtu.be/qAF1NjEVHhY?si=CKShLFYuOz0xbi1N

Krish Naik Complete RAG Crash Course With Langchain In 2 Hours https://youtu.be/o126p1QN_RI?si=7xC5H47A3iuu52RK

pixegami Python RAG Tutorial (with Local LLMs): AI For Your PDFs https://youtu.be/2TJxpyO3ei4?si=zPNEsAFWWy5Cenzq

UTVerse https://utverse.tennessee.edu/

Technologies, Cuantum. Natural Language Processing with Python Updated Edition: From Basics to Advanced Projects: Mastering Text Analysis, Machine Learning Models, and Chatbot … (Mastering the AI Revolution Book 4) (p. 1). (Function). Kindle Edition.

Tunstall, Lewis; Werra, Leandro von; Wolf, Thomas. Natural Language Processing with Transformers, Revised Edition (p. 4). (Function). Kindle Edition.

"The Complete Guide to Ollama: Run Large Language Models Locally." DEV Community, 16 Feb. 1970, dev.to/ajitkumar/the-complete-guide-to-ollama-run-large-language-models-locally-2mge. Accessed 28 Mar. 2026.